

IoT-Based Automated Tropical Terrarium Using ESP32

1st Kishor Tarafder

*Dept. of Computer Science and
Engineering
American International University-
Bangladesh
Dhaka, Bangladesh
23-54520-3@student.aiub.edu*

2nd MD. Shakhawat Hossen

*Dept. of Computer Science and
Engineering
American International University-
Bangladesh
Dhaka, Bangladesh
23-54483-3@student.aiub.edu*

3rd Tahomina Era

*Dept. of Computer Science and
Engineering
American International University-
Bangladesh
Dhaka, Bangladesh
23-55027-3@student.aiub.edu*

4th Avishek Saha

*Dept. of Computer Science and
Engineering
American International University-
Bangladesh
Dhaka, Bangladesh
23-54508-3@student.aiub.edu*

5th Jannatun Nur Deba

*Dept. of Computer Science and
Engineering
American International University-
Bangladesh
Dhaka, Bangladesh
23-54545-3@student.aiub.edu*

6th MD. Alfaz Uddin

*Dept. of Computer Science and
Engineering
American International University-
Bangladesh
Dhaka, Bangladesh
23-55557-3@student.aiub.edu*

Abstract—Tropical terrarium plants require a narrow band of humidity, temperature, soil moisture and illumination that is difficult to maintain manually, while commercial plant controllers are either open-loop timers or cloud-dependent monitors. This paper presents the design, implementation and experimental evaluation of a self-contained terrarium controller built around the ESP32 microcontroller. The system measures air temperature, relative humidity and pressure, illuminance, soil moisture at two points, tray leakage and reservoir level, and drives two ultrasonic mist makers, an exhaust fan, a grow light and an alarm through opto-isolated MOSFET stages. Control runs entirely on the device: a rules engine combines a burst-and-soak watering state machine with a daily watering cap, humidity hysteresis with bounded burst length and cool-down, a photoperiod-gated lighting rule, and hardware interlocks for an empty reservoir and leakage. Fourteen control parameters are adjustable at run time through a web dashboard served over Wi-Fi or a local hotspot and through a USB serial interface that also provisions network credentials. Experiments on a prototype calibrated two capacitive probes by two-point measurement (usable spans of 1380 and 1415 ADC counts), showed that the opto-isolated switching stage requires a 5 V active-low drive rather than 3.3 V logic, confirmed that flyback protection is necessary for the fan, and verified wireless control at a link strength of about -53 dBm. The controller operates without internet access and keeps the enclosure within its target band without daily attention.

Keywords—ESP32, Internet of Things, terrarium, closed-loop control, hysteresis, environmental monitoring

I. INTRODUCTION

A terrarium is a small, largely enclosed ecosystem in which tropical plants such as ferns, mosses, nerve plants (*Fittonia*) and orchids are cultivated. Inside the enclosure the plants sustain their own water cycle: moisture evaporates from the substrate, condenses on the walls and returns, so the system can remain stable for weeks when the balance is correct. These species require 70 to 90 percent relative humidity, a temperature of about 22 to 28 degrees Celsius, evenly moist but never waterlogged soil, and roughly twelve hours of light

per day. In a tropical urban home such conditions are hard to hold by hand: summer temperatures in Dhaka reach 33 to 36 degrees Celsius, air conditioning and the dry season lower indoor humidity within hours, and the illuminance near a window varies from a few hundred lux to several thousand. Manual care fails silently, because the owner notices the damage only after the plant has declined.

Existing products fall into two groups. Timer-based misting systems act without feedback: they mist on schedule even when the enclosure is saturated and continue when the reservoir is empty, which destroys the ultrasonic transducer. Cloud-connected plant monitors close the loop on at most one variable, depend on an internet connection and a vendor service, and offer no hardware protection against an empty reservoir or a leaking tray. Published research systems, reviewed in Section II, are mostly monitoring oriented, use a single threshold per variable and rely on cloud platforms for remote control.

This paper presents a low-cost controller that senses, decides and acts locally on the enclosure, closes the loop on every relevant variable at once, fails safe, and remains fully operable without internet access. The main contributions are: (1) a control design that combines a burst-and-soak watering state machine with a daily cap, humidity hysteresis with bounded bursts and cool-down, photoperiod-gated lighting and hardware interlocks; (2) a run-time configurable, local-first architecture in which a phone dashboard and a USB serial interface share one command layer; (3) an experimentally established switching-stage design, namely a 5 V active-low drive of opto-isolated MOSFET channels with flyback protection; and (4) a calibrated prototype operating with a live plant.

The remainder of the paper is organized as follows. Section II reviews related work. Section III describes the system design and Section IV its implementation. Section V reports the experimental results and discussion, and Section VI concludes the paper.

II. RELATED WORK

Oguntosin et al. [1] developed an ESP32-based greenhouse monitoring and control system that reads temperature, humidity, a light-dependent resistor and a resistive soil-moisture sensor and switches a bulb, a fan and a pump whenever a parameter crosses a fixed threshold; readings are stored in a database and shown on a web application. The work demonstrates the basic sense-decide-act loop on the same microcontroller family, but thresholds are compiled into the firmware, control is single-sided without hysteresis, and there is no protection against an empty water source.

Correa-Quiroz et al. [2] reported an ESP32 drip-irrigation and climate-monitoring system for greenhouses with DHT11, capacitive soil-moisture and ultrasonic water-level sensors, an LCD and the Arduino Cloud. Field tests showed a 35 percent reduction in water use, and the system offers automatic and manual modes. Remote control, however, depends on a cloud service. Abu Snehneh and Shabaneh [3] built a hydroponics monitor on the ESP32 with TDS, pH, water-level and temperature sensors in which pumps are actuated automatically or from the Blynk application; again the interface is cloud-hosted and the actuation carries no time-based limits.

Mellit et al. [4] designed a remote greenhouse-monitoring prototype around a NodeMCU that measures air temperature, relative humidity, capacitive soil moisture, light intensity and carbon dioxide, publishes them to a web page, sends GSM alerts and adds a Raspberry Pi camera with a convolutional neural network for disease detection. The breadth of sensing is close to the present work, while the two-processor architecture raises the cost well above what a home terrarium justifies. Simo et al. [7] presented a low-cost IoT greenhouse monitoring device for a farmer's decision making; like most surveyed work it is monitoring oriented and leaves actuation to the user.

Garcia et al. [5] surveyed sensors and IoT nodes for smart irrigation and concluded that low-cost capacitive soil-moisture probes are adequate for irrigation scheduling if calibrated in the target medium, and that they outlast resistive probes whose electrodes corrode in wet soil; this motivated the calibration procedure of Section III-B. The BME280 provides humidity, temperature and pressure in one I2C device with a specified humidity accuracy of about 3 percent RH [9], sufficient for a 15 percent hysteresis band, and the BH1750 reports illuminance directly in lux [10]. Hercog et al. [6] documented the design of ESP32-based IoT devices and evaluated the ESP32 as a dual-core, Wi-Fi-capable and inexpensive platform [8], which supports its selection here.

Table I summarizes the comparison. The gap addressed by this work is the combination of multi-variable closed-loop control with hysteresis, hardware safety interlocks, run-time adjustable parameters, a manual mode with automatic time-outs, and local-first operation that stays controllable from a phone without any internet or cloud service.

TABLE I. COMPARISON WITH RELATED SYSTEMS

Work	Platform	Sensed variables	Actuation	Offline control	Safety interlocks
[1]	ESP32	T, RH, light, soil	Bulb, fan, pump	No	No

Work	Platform	Sensed variables	Actuation	Offline control	Safety interlocks
[2]	ESP32	T, RH, UV, soil, level	Drip valve	No (cloud)	Level sensing only
[3]	ESP32	T, pH, TDS, level	Pumps	No (Blynk)	No
[4]	NodeMCU + RPi	T, RH, soil, light, CO2	Irrigation, fan	No	No
[7]	IoT node	Environment, electrical	None	No	No
This work	ESP32	T, RH, P, lux, 2x soil, leak, level	2 mist, fan, light, buzzer	Yes (hotspot + USB)	Float, leak, daily cap, time-outs

III. SYSTEM DESIGN

A. Architecture

The system consists of a sensing stage, a controller and an actuation stage arranged around a single ESP32 module. The sensing stage comprises a BME280 (temperature, relative humidity, pressure) and a BH1750 (illuminance) on the I2C bus, two capacitive soil-moisture probes and a leak probe on the 12-bit analog inputs, and a float switch on a digital input. The actuation stage consists of two 4-channel opto-isolated MOSFET boards: one on a 5 V rail for the two ultrasonic mist makers and the alarm, the other on 12 V for the fan and the LED grow light. A 12 V adapter feeds a fused buck converter that provides the 5 V rail, and the ESP32's on-board regulator supplies 3.3 V to the sensors. Table II lists the components. Every two seconds the controller samples all inputs, evaluates the control rules of Section III-C, applies the safety interlocks and writes the outputs; clients on Wi-Fi or USB observe and command the system through a shared command layer described in Section IV.

TABLE II. SYSTEM COMPONENTS

Component	Interface / rating	Function
ESP32 DevKit V1	CP2102 USB, Wi-Fi	Controller, web server, USB interface
GY-BME280	I2C 0x76, 3.3 V	Air temperature, RH, pressure
BH1750FVI	I2C 0x23, 3.3 V	Illuminance (lux)
Capacitive soil probe x2	ADC, 3.3 V	Soil moisture at two points
Leak probe	ADC, 3.3 V	Drip-tray leak detection
Float switch	GPIO, pull-up	Reservoir empty interlock
MOSFET board x2	Opto-isolated, 60N03	5 V and 12 V load switching
Mist maker x2	DC 5 V, 1 A driver	Watering, humidity
Fan	12 V, 0.08 A	Exhaust ventilation
LED strip 5050	12 V, IP65	Grow light, PWM dimmed
Passive buzzer	2.7 kHz tone	Alarm
12 V adapter, LM2596S buck	12 V in, 5 V out	Power supply

B. Sensing and Calibration

Soil moisture is derived from the raw ADC reading of each capacitive probe by the two-point linear calibration

$$M(\%) = 100 (ADC_{dry} - ADC_{raw}) / (ADC_{dry} - ADC_{wet}) \quad (1)$$

where ADC_{dry} and ADC_{wet} are the readings obtained with the probe in air and fully submerged in water, and the result is clamped to the range 0 to 100 percent. The two probes are averaged for the watering decision so that a single dry pocket does not trigger a burst. The probes and the leak probe are connected to input-only pins of the first ADC unit, because the second unit is unavailable while the Wi-Fi radio is active. The leak probe is treated as wet above a fixed count, and the float switch is read through an internal pull-up with a configurable empty state. Three consecutive failed humidity readings are treated as a sensor fault that switches the humidity mist off rather than on, since a dropped I2C transaction is not evidence of dry air.

C. Control Strategy

Watering follows a burst-and-soak state machine with the states IDLE, RUN and SOAK. When the average moisture falls below the dry threshold and the reservoir is not empty, the first mist maker runs for one burst of duration t_{run} ; the controller then waits a soak period t_{soak} for the water to spread through the substrate before the soil is judged again. A hard cap bounds the total misting time per day even if a probe fails in the dry direction; the daily total is reset at midnight when a network clock is available and every 24 h of uptime otherwise.

Humidity control uses a hysteresis band: the second mist maker turns on when RH falls below RH_{low} and off when RH exceeds RH_{high} , subject to a maximum burst length and a cool-down between bursts. The band avoids the on-off chatter of a single threshold, and the cool-down allows the sensor to register the mist just released before a new decision is made. The grow light follows a photoperiod window gated by illuminance: it is switched on below E_{on} and off above E_{off} inside the window and is always off outside it; without a valid clock the illuminance gate acts alone. The exhaust fan turns on above 32 degrees Celsius and off below 29. Three interlocks override all rules: an empty reservoir disables both mist channels, a wet leak probe raises the alarm, and any manual mist command is cancelled after a fixed time. Table III lists the parameters and their default values; all are adjustable at run time.

TABLE III. CONTROL PARAMETERS AND DEFAULT VALUES

Parameter	Default	Role
Soil dry threshold	35 %	watering below this average moisture
RH low / high	75 / 90 %	humidity mist on / off
Illuminance on / off	800 / 2000 lx	grow light on / off
Photoperiod	07 to 19 h	lighting window
Light duty	220 / 255	PWM brightness
Burst / soak	90 s / 20 min	watering cycle
Daily cap	30 min	maximum watering per day
Humidity burst / cool-down	300 s / 180 s	bounded misting
Manual time-out	10 min	manual mist auto-off

D. Power and Switching Stage

The power architecture has three rails. The 12 V rail from the adapter, protected by a 3 A fuse, feeds the buck converter and the 12 V loads. The 5 V rail from the buck converter feeds the ESP32, the mist driver modules and the inputs of the opto-

couplers on both switching boards. The 3.3 V rail feeds the sensors only. Bulk capacitors at the buck output and at each mist driver absorb the inrush of a transducer so that the logic supply does not dip.

The opto-coupled inputs of the MOSFET boards require about 5 V to switch, which the 3.3 V logic of the ESP32 cannot supply directly. Each channel is therefore driven active-low: the input's positive terminal is tied to the 5 V rail and its return is pulled low by an ESP32 pin, so the microcontroller only sinks the small opto-LED current. The firmware inverts every output accordingly, including the PWM duty of the grow light, and writes all outputs to the off state before and after the pins are configured at boot, so that no channel can pulse during a reset. Because the fan is an inductive load, a 1N5819 Schottky diode is placed across it with the cathode to the positive lead; the ultrasonic transducers are piezoelectric loads and need no diode.

IV. IMPLEMENTATION

A. Firmware

The firmware is written in C++ for the Arduino ESP32 core. A single non-blocking loop samples the sensors every 2 s, evaluates the rules in automatic mode or the manual guards in manual mode, and calls one output function that applies the interlocks and writes the pins. The fourteen control parameters are stored in non-volatile memory, are loaded at boot with compiled defaults as fallback, and can be changed by any client at run time. A shared command layer implements mode changes, output commands and parameter writes exactly once; both the HTTP handlers of the embedded web server and the USB serial parser call it, so wired and wireless control cannot diverge. Status strings are assembled in fixed buffers, and time comparisons use signed differences so that the 49-day millisecond counter wrap cannot freeze a cool-down.

B. Interfaces and Provisioning

The embedded web server exposes a dashboard page and a small JSON API for status, outputs, mode, parameters and network credentials. The USB serial interface accepts the same commands as line-oriented text at 115200 baud and marks every reply with a prefix so that a host can separate replies from the boot log. Network credentials saved through either interface take precedence over compiled placeholders. If no network is configured or the connection fails, the board raises its own access point immediately and keeps retrying the saved network in the background every 90 s, while the clock is re-synchronized from NTP every 5 min until it succeeds. A host application was written to install the USB driver and the tool-chain, flash the firmware, provision credentials over the cable and exercise each output channel manually; it was used for the experiments in Section V.

C. Prototype

Fig. 1(a) shows the bench setup used for the electrical measurements, with the ESP32 and sensors on a breadboard, the 5 V switching board and buck converter, and the mist maker, leak probe and buzzer in a plastic tub. Fig. 1(b) shows the prototype operating with a live *Fittonia* plant in a transparent enclosure: a soil probe is inserted in the substrate, the air sensor and the fan are mounted inside, and the

controller is housed alongside. A digital multimeter was used for the rail voltages and switching checks, and a serial terminal for the firmware trace.

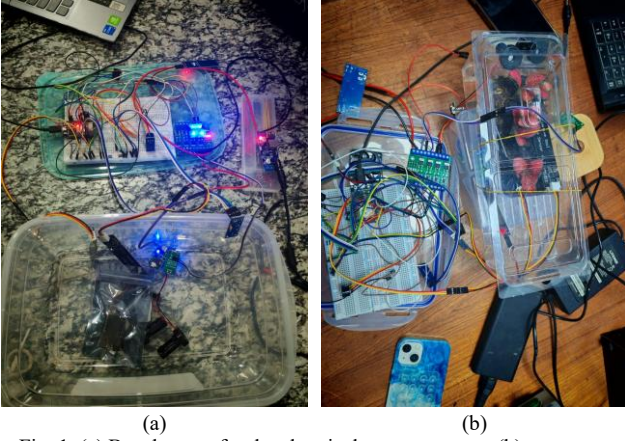


Fig. 1. (a) Bench setup for the electrical measurements; (b) prototype operating with a live *Fittonia* plant.

V. RESULTS AND DISCUSSION

A. Sensor Calibration

Table IV gives the two-point calibration of the soil probes and Fig. 2 the resulting transfer curves. Both probes read within a few counts of their end points when held in air or in water and responded to wetting within seconds. The usable spans of 1380 and 1415 counts correspond to a resolution of 0.072 and 0.071 percent moisture per count, two orders of magnitude finer than the 35 percent watering threshold, so quantization does not influence the decision. A disconnected probe floats near 2490 counts, which the firmware interprets as dry. The I2C scan reported the BME280 at address 0x76 and the BH1750 at 0x23; the BME280 read room conditions of about 67 percent RH and responded to breath or mist within seconds, and the BH1750 tracked changes in room light.

TABLE IV. TWO-POINT SOIL PROBE CALIBRATION (12-BIT ADC)

Probe	Dry (air)	Wet (water)	Span
Probe 1	2450	1070	1380
Probe 2	2485	1070	1415

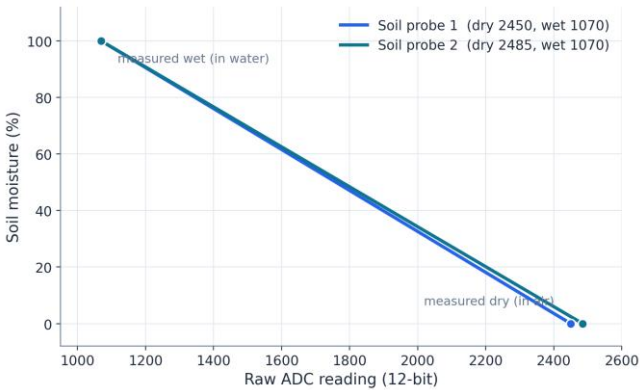


Fig. 2. Measured soil-moisture calibration transfer curves.

B. Electrical Characterization

Table V compares design values with measurements. The adapter delivered 12.0 V and the buck converter 5.2 V under

load, 4 percent above the 5.0 V design value and within the tolerance of the mist drivers and the ESP32 regulator. Driving a switching-board input from a 3.3 V pin produced only a faint input indication and no switching, with about 0.6 to 0.8 V measured across the opto-coupler; the expectation that 3.3 V logic would drive the inputs was therefore rejected and the active-low 5 V arrangement of Section III-D adopted, after which the channel switched cleanly and the mist maker ran on command. Switching the fan without a flyback diode caused the ESP32 to brown out and reset, whereas the same channel switched reliably with the load removed, which identifies the inductive transient as the cause and confirms the diode as the remedy.

TABLE V. DESIGN VALUES VERSUS MEASUREMENTS

Quantity	Design	Measured
Adapter output	12.0 V	12.0 V
5 V rail (buck output)	5.0 V	5.2 V
ESP32 3.3 V rail	3.3 V	3.2 V
Opto input, 3.3 V drive	switching	0.6 to 0.8 V, no switching
Opto input, 5 V active-low	5 V / 0 V	5.2 V / 0 V, switching
Fan without flyback diode	no effect	controller reset
Wi-Fi link strength	> -70 dBm	-53 dBm

C. Control Behaviour and Connectivity

With the humidity band at its defaults, the humidity channel engaged as soon as the measured RH of 67 percent lay below the 75 percent lower threshold and released only when the upper threshold was crossed or the burst limit expired, as the control law prescribes. The watering state machine executed its burst and soak phases on command, and the alarm produced a clean 2.7 kHz tone from the manual control. On the network side the board joined the local access point with a received signal strength of about -53 dBm, and the dashboard was reachable from a phone. Credentials pushed over USB caused the board to restart, join the new network and be rediscovered automatically; with no router present the board raised its own access point and the dashboard remained reachable. A parameter changed from the host was read back correctly after a power cycle. During development an apparent loss of two thirds of dashboard requests was traced not to the radio but to a firmware fault on every status request caused by a mismatched format string; after the correction every request was served.

D. Discussion

The results confirm the two design decisions that were not obvious a priori: opto-isolated switching boards intended for 5 V logic must be driven active-low from a 3.3 V microcontroller, and inductive loads need flyback protection even at fan currents of 80 mA, because the transient couples into the shared supply and resets the controller. The hysteresis and burst-limited control produced the expected non-chattering behaviour, and the local-first architecture met the requirement of full operability without internet access. The evaluation has limits. The measurements were made on the 5 V stage of a prototype with a temporary enclosure; a long-duration unattended run against a reference hygrometer, which would allow a quantitative comparison of the humidity

trace with the control model, remains to be performed. Humidity is sensed at one point, so gradients inside a larger enclosure are not observed, and the air sensor must be shielded from direct mist. Finally, the transducers must never run dry, which the float interlock guarantees only when the switch is installed in the reservoir.

VI. CONCLUSION AND FUTURE WORK

A self-contained closed-loop terrarium controller was designed, implemented and evaluated on a single ESP32. Seven sensed quantities drive mist, ventilation, lighting and alarm outputs through isolated switching stages under a rules engine that combines a watering state machine, humidity hysteresis, bounded burst times, a daily cap and hardware interlocks. The soil probes were calibrated, the switching stage was characterized and corrected to an active-low 5 V drive with flyback protection, and control was demonstrated over Wi-Fi, a local access point and USB, with all parameters adjustable at run time and no dependence on internet services. Future work will integrate a purpose-built enclosure with the 12 V lighting and ventilation stage, perform a long-duration run against a reference hygrometer, add on-device data logging, and evaluate thermoelectric cooling for the hottest months.

REFERENCES

- [1] V. W. Oguntosi, C. Okeke, E. Adetiba, A. Abdulkareem, and J. O. Olowoloni, "IoT-based greenhouse monitoring and control system," *Int. J. Comput. Digit. Syst.*, vol. 14, no. 1, 2023, doi: 10.12785/ijcds/140137.
- [2] J. J. Correa-Quiroz, M. A. Toribio-Barrueto, and C. Castro-Vargas, "IoT system with ESP32 for smart drip irrigation and climate monitoring in greenhouses," *Emerg. Sci. J.*, vol. 9, no. 3, pp. 1133–1157, Jun. 2025, doi: 10.28991/esj-2025-09-03-01.
- [3] A. Abu Sneineh and A. A. A. Shabaneh, "Design of a smart hydroponics monitoring system using an ESP32 microcontroller and the Internet of Things," *MethodsX*, vol. 11, Art. no. 102401, 2023, doi: 10.1016/j.mex.2023.102401.
- [4] A. Mellit, M. Benghanem, O. Herrak, and A. Messalaoui, "Design of a novel remote monitoring system for smart greenhouses using the Internet of Things and deep convolutional neural networks," *Energies*, vol. 14, no. 16, Art. no. 5045, Aug. 2021, doi: 10.3390/en14165045.
- [5] L. García, L. Parra, J. M. Jiménez, J. Lloret, and P. Lorenz, "IoT-based smart irrigation systems: An overview on the recent trends on sensors and IoT systems for irrigation in precision agriculture," *Sensors*, vol. 20, no. 4, Art. no. 1042, Feb. 2020, doi: 10.3390/s20041042.
- [6] D. Hercog, T. Lerher, M. Truntiĉ, and O. Težak, "Design and implementation of ESP32-based IoT devices," *Sensors*, vol. 23, no. 15, Art. no. 6739, Jul. 2023, doi: 10.3390/s23156739.
- [7] A. Simó, S. Dziřac, G. E. Badea, and D. Meianu, "Smart agriculture: IoT-based greenhouse monitoring system," *Int. J. Comput. Commun. Control*, vol. 17, no. 6, Art. no. 5039, Dec. 2022, doi: 10.15837/ijccc.2022.6.5039.
- [8] Espressif Systems, "ESP32 series datasheet," v4.x, Shanghai, China. [Online]. Available: <https://www.espressif.com/en/support/documents/technical-documents>
- [9] Bosch Sensortec, "BME280: Combined humidity and pressure sensor," Datasheet BST-BME280-DS002, Reutlingen, Germany.
- [10] ROHM Semiconductor, "BH1750FVI: Digital 16-bit serial output type ambient light sensor IC," Datasheet, Kyoto, Japan.